Informe Comparativo: Simplex vs Gran M

Grupo 3 · IO1

Noviembre 2026

Daniel Chavarro (P1) · Jorge Méndez (P2)

Nicolás Velazco (P3) · Santiago Vargas (P4)

---

1. Objetivo

Comparar cuantitativamente el **método Simplex estándar** y el **método de la Gran M** sobre los cuatro casos de prueba del proyecto (incluido el caso real de Procafecol), midiendo:

- **Número de iteraciones (pivotes)**
- **Tiempo de cómputo** (promedio de 30 ejecuciones, ms)
- **Tamaño de la tabla** (filas × columnas de variables)
- **Cuándo conviene** cada método según el tipo de restricciones

Y verificar el resultado del caso real contra la solución obtenida con **Excel Solver**.

2. Motor y configuración

Parámetro	Valor
Aritmética	Exacta — `fractions.Fraction` + `FracM` (M simbólica)
Criterio de entrada	Mayor $C_j - Z_j > 0$ (max); menor $C_j - Z_j < 0$ (min)
Criterio de salida	Razón mínima positiva
Límite de iteraciones	200
Entorno	Python 3.13.7, Windows 11 (tiempo indicativo)

3. Resultados por caso

3.1 Caso real — Plan de producción Procafecol (`bakery-real`)

5 variables (líneas de café), **13 restricciones** (5 >=, 8 <=):

Método	Estado	Pivotes	Tiempo (ms)	Filas tabla	Cols tabla	Valor Z
Simplex	`not_admissible`	—	1.32	—	—	—
Gran M	`optimal`	**9**	**47.87**	13	23	**135 857/36 ≈ 3 773.81**

- El Simplex **no puede resolver este caso**: tiene restricciones >= que requieren variables artificiales.
- Gran M introduce **5 excesos + 5 artificiales** → tabla 13 × 23.
- Las 5 artificiales **salen de la base** en los 9 pivotes, confirmando factibilidad.

3.2 Contenedores en la nube — `simplex-grupo1`

3 variables, **4 restricciones** <=:

Método	Estado	Pivotes	Tiempo (ms)	Filas tabla	Cols tabla	Valor Z
Simplex	`optimal`	**3**	**1.94**	4	7	**1 575**
Gran M	`optimal`	**3**	2.12	4	7	**1 575**

- Ambos métodos convergen en el mismo número de pivotes con idéntico óptimo.
- Gran M agrega **0 artificiales** (todas las restricciones son <=), la tabla es igual.
- El overhead de Gran M (~9 %) se debe al manejo de la representación `FracM` simbólica.

3.3 PL infactible — `test-infeasible`

1 variable, restricciones $x_1 \leq 2$ y $x_1 \geq 5$ (región vacía):

Método	Estado	Pivotes	Tiempo (ms)	Filas tabla	Cols tabla
Simplex	`not_admissible`	—	0.013	—	—
Gran M	`infeasible`	1	0.51	2	4

- Gran M detecta la infactibilidad en 1 pivote: la artificial a_1 queda en base con valor $3 > 0$.

3.4 PL no acotado — `test-unbounded`

2 variables, $x_1 - x_2 \leq 2$, maximizar $x_1 + x_2$:

Método	Estado	Pivotes	Tiempo (ms)	Filas tabla	Cols tabla
Simplex	`unbounded`	1	0.35	1	3
Gran M	`unbounded`	1	0.30	1	3

4. Tabla consolidada

Caso	Tipo	Vars	Restricc.	Simplex	Gran M	Δ Pivotes	Δ Tiempo
Procafecol	Solo M	5	13 (>=, <=)	No aplica	9 pivotes	—	—
Contenedores	Solo S	3	4 (<=)	3 pivotes	3 pivotes	0	+9 %
Infactible	Solo M	1	2 (<=, >=)	No aplica	1 pivote	—	—
No acotado	S = M	2	1 (<=)	1 pivote	1 pivote	0	−15 %

S = puede usar Simplex; M = requiere Gran M (tiene >= o =).

5. Análisis comparativo

5.1 Tamaño de la tabla

La tabla del **Simplex** tiene `n + m + 1` columnas (n variables originales, m holguras, 1 RHS).

La tabla de **Gran M** puede tener hasta $n + m_{slack} + m_{excess} + m_{art} + 1$ columnas.

En Procafecol: 5 + 8 + 5 + 5 + 1 = **24 columnas** (23 de variables + RHS).

Cuando todas las restricciones son <=, ambas tablas son **idénticas**: Gran M no introduce columnas extra.

5.2 Iteraciones

- Para problemas puramente <=: el número de pivotes es **idéntico** entre ambos métodos.
- Para problemas con >= o =: Gran M necesita pivotes adicionales para expulsar artificiales.

5.3 Tiempo de cómputo

Componente	Simplex	Gran M
Build inicial	Fraction puro	+ overhead FracM
Por pivote	O(nm) Fraction	O(nm) Fraction + comparaciones FracM
Overhead medido (caso <=)	—	~9 %

5.4 ¿Cuándo conviene cada método?

Situación	Método recomendado	Razón
Todas las restricciones <= y RHS ≥ 0	**Simplex**	Tabla mínima, sin artificiales, sin overhead M
Al menos una >= o =	**Gran M**	Único método admisible; detecta infactibilidad
RHS negativo en alguna restricción	**Gran M**	Normaliza multiplicando por −1 automáticamente
Objetivo minimización	Ambos	Mismo algoritmo, sentido invertido en $C_j - Z_j$

Regla práctica del grupo: usar Gran M cuando haya >= o =; Simplex cuando todo sea <=.

6. Verificación con Excel Solver

El caso real (Procafecol, `bakery-real`) fue verificado con **Excel Solver (método Simplex LP)**.

Resultado comparado

Variable	Valor Excel Solver	Valor Gran M (motor exacto)	Diferencia
Tradicional	60.000	60	0
Premium Balanceado	35.000	35	0
Orgánico	35.000	35	0
Origen Regional	10.278	370/36 ≈ 10.278	< 0.001
Mujeres Cafeteras	10.500	21/2	0
Z (utilidad, M$)	**3 773.806**	**135 857/36 = 3 773.8056**	< 0.001

> **Nota.** La diferencia < 0.001 es error de redondeo de punto flotante de Excel. El motor usa aritmética exacta (`Fraction`), cuyo valor exacto es **Z = 135 857/36**.

Restricciones activas (holgura = 0)

Las restricciones activas definen el vértice óptimo:

1. **Café verde total** (≤ 236 ton)
2. **Mínimo retail** (≥ 95 ton: Tradicional + Premium)
3. **Mínimo 25 % especial** (participación de líneas especiales)
4. **Máximo Orgánico** (≤ 35 ton)
5. **Mínimo Mujeres Cafeteras** (≥ 10.5 ton)

> **Importante.** Excel Solver y el motor del proyecto coinciden. La verificación es satisfactoria.

7. Conclusiones

1. **Gran M es el único método universal**: admite <=, >=, = y RHS negativos. Su overhead es < 15 % en problemas de tamaño del curso.
2. **Simplex puro es más eficiente** cuando todas las restricciones son <= con RHS ≥ 0: tabla mínima, sin artificiales, sin overhead simbólico.
3. **Para el caso real Procafecol**, Gran M resuelve en 9 pivotes y obtiene Z = 135 857/36 ≈ **$3 773.81 M/mes**. Coincide con Excel Solver.
4. **Las 5 restricciones activas** son los cuellos de botella de producción. El análisis de sensibilidad (precios sombra) sobre estas restricciones revelaría el valor marginal de relajarlas.
5. **Recomendación operativa:** dado que el problema real siempre tendrá restricciones >=, el sistema debe ofrecer Gran M como método por defecto.

8. Declaración de uso de IA

Este informe fue desarrollado con asistencia de **Antigravity (Google DeepMind)** como herramienta de pair programming. El análisis matemático, los datos de Excel Solver y la interpretación de resultados son responsabilidad del Grupo 3.